

Laboratorio 8: Respuestas

Deep Learning y Sistemas Inteligentes - CC3092

José Antonio Mérida Castejón

4 de octubre de 2026

El código, las salidas y las gráficas de todos los tasks ya están en el notebook `Notebook.ipynb`; este documento solo contiene las respuestas a las preguntas.

Task 2.2

(a) ¿Por qué greedy (config. A) repite o entra en ciclos?

Greedy elige $w_{t+1} = \arg \max_i z_i$. Es decir, es una función determinista que dado el mismo contexto siempre produce el mismo carácter y descarta completamente el resto. El comportamiento repetitivo se puede observar en las 5 muestras idénticas (al ser determinista) que caen en un ciclo de repeticiones de “the stay”, “the see”, “the stand”.

```
Why, there the stay the see the stay the stand
That the stay the stay the stand the stay the seem
Than the stay the seem the seem the seem and the seembers
The can the seems and the seember the seemb
```

Esto ocurre porque, aunque en muchas posiciones la distribución del siguiente carácter está repartida entre varias opciones plausibles, greedy descarta toda esa masa y se queda siempre con la opción más probable dado el contexto. Esta suele ser una continuación muy común, por ejemplo, palabras funcionales como **the**. Al ser determinista, basta con que el contexto llegue a un estado donde el **argmax** lleve de vuelta a una secuencia ya vista para que el texto caiga en un ciclo.

(b) Config. B (top-k, $k = 5$, $\tau = 0.7$) vs. C (top-k, $k = 50$, $\tau = 1.0$)

Con temperatura, la probabilidad de cada carácter es

$$P(w_i \mid ctx) = \frac{\exp(z_i/\tau)}{\sum_j \exp(z_j/\tau)}$$

En top-k, la suma del denominador solo incluye los k caracteres con mayor logit. El resto tiene probabilidad 0.

El parámetro k fija cuántos caracteres pueden muestrearse en cada paso. Con $k = 5$ el modelo solo elige entre los cinco más probables, así que casi todo lo que genera son palabras reales y frecuentes. Por ejemplo, B1 produce “The death shall now and to my thank he streate, And the come a see of

the cale have at strange.” A cambio, el texto repite mucho las palabras funcionales, como “be be” en B5, “to thee come with thee world” en B2 o “the wear the will” en B1. Con $k = 50$ casi no hay recorte, ya que el vocabulario tiene 65 caracteres. Entran caracteres poco probables y el texto se vuelve más variado pero también más ruidoso. Aparecen palabras inventadas como “withipinus” y “nisur” en C1 o “seepinated’s” y “petteignness” en C5, y también más signos de puntuación.

La temperatura divide los logits antes del softmax, así que cambia qué tan separadas quedan las probabilidades. Si dos caracteres tienen logits que difieren en 1, el ratio entre sus probabilidades es $e^{1/\tau}$. Con $\tau = 1.0$ ese ratio es $e^1 \approx 2.7$, y con $\tau = 0.7$ es $e^{1/0.7} \approx 4.2$. Es decir, con $\tau = 0.7$ el carácter más probable gana todavía más peso frente a los demás y la distribución se concentra. Con $\tau = 1.0$ se usa la distribución original del modelo, que es más plana y le da más peso a las opciones poco probables.

Hay una limitación importante. B y C cambian ambos parámetros a la vez, así que estas muestras no permiten aislar el efecto de k y el de τ por separado. Lo que se observa es consistente con lo que predice la fórmula, pero no lo prueba por sí solo.

En cuanto a coherencia y creatividad, B es la más coherente a nivel de palabras, porque casi todas existen en inglés. Esto tiene sentido porque los cinco caracteres más probables suelen ser continuaciones razonables, mientras que el carácter número 30 o 50 casi nunca lo es. En C cada uno de esos caracteres tiene probabilidad baja, pero juntos suman una masa que no es despreciable, sobre todo con $\tau = 1.0$. A lo largo de 200 caracteres es casi seguro que en algún momento se elija uno de ellos, y basta un solo carácter malo para romper una palabra. Además, ese error queda en el contexto y afecta las predicciones siguientes. Por eso C es la más diversa y creativa, pero produce muchas pseudo-palabras. B en cambio, no forma frases con sentido y es repetitiva, porque siempre se mueve entre las mismas opciones seguras.

(c) Config. E (top-p = 0.95, $\tau = 1.0$): momentos de seguridad e incertidumbre

El tamaño del núcleo depende de la forma de la distribución en cada paso. Si el modelo está seguro, un solo carácter puede acumular casi toda la probabilidad y el núcleo es muy pequeño. Si está inseguro, la probabilidad está repartida y hacen falta muchos caracteres para llegar a 0.95.

Para respaldar esto con datos, la celda anterior genera una muestra adicional con la configuración E y registra en cada paso la probabilidad máxima $p_{\text{máx}}$ y el tamaño del núcleo. Los números se refieren a esa muestra y no a E1 a E5, pero el mismo comportamiento se ve en las cinco muestras.

El modelo está muy seguro en posiciones de puntuación, formato y continuaciones obvias de palabra. En todos estos casos el núcleo tiene un solo carácter. Después de los dos puntos que cierran el nombre de un personaje, el siguiente carácter es un salto de línea con $p_{\text{máx}} = 0.99$. Esto se ve en todas las muestras, donde nombres como “RARCHIS:”, “MARDIUS:” o “KING RICHARD II:” siempre van seguidos de un salto de línea. Algo parecido pasa después de un punto, donde viene un salto de línea con $p_{\text{máx}} = 0.98$, y después de una coma o al final de una palabra, donde viene un espacio con $p_{\text{máx}}$ entre 0.95 y 0.96. También hay continuaciones casi forzadas dentro de las palabras. Después de una “i” el modelo elige “n” con $p_{\text{máx}} = 0.95$, y después de una “n” elige “g” con $p_{\text{máx}} = 0.97$. Esto es consistente con la terminación “-ing”, que es muy frecuente en el corpus y que el modelo incluso aplica a palabras inventadas como “Thereing” o “childing”. En todos estos casos el corpus es casi determinista, así que la distribución se concentra en una sola opción.

El modelo está inseguro en las fronteras de palabra y de línea. Los casos más inciertos son casi todos la primera letra de una palabra, justo después de un espacio. Ahí el núcleo tiene entre 24 y

30 de 65 caracteres y $p_{\text{máx}}$ está entre 0.12 y 0.23, porque elegir qué palabra viene es una decisión muy abierta. Al empezar una línea nueva pasa algo parecido, con un núcleo de 24 caracteres y $p_{\text{máx}}$ entre 0.21 y 0.31. El modelo no sabe qué personaje o qué palabra abre la línea, y es justo ahí donde aparecen los nombres inventados de las muestras, como “RARCHIS” o “KICHARD ING HERMARD III”. Una vez que la palabra empezó, las opciones se reducen y la incertidumbre baja.

En total, el núcleo tuvo en promedio unos 9.1 caracteres, con valores entre 1 y 30. Esto muestra la adaptatividad que describe el enunciado. Un top-k fijo usaría siempre el mismo número de candidatos, mientras que top-p usa uno solo cuando el modelo está seguro y hasta 30 cuando no lo está.

Task 3.2

(a) Similitud coseno entre contextos distintos de la misma palabra (BERT)

Palabra	Contexto 1	Contexto 2	Coseno BERT
banco	institución financiera	asiento	0.5753
banco	institución financiera	cardumen	0.6603
banco	asiento	cardumen	0.6496
copa	trofeo	recipiente	0.5667
vela	de barco	cirio	0.7999

(b) Word2Vec vs. BERT

Palabra	Contexto 1	Contexto 2	Coseno BERT	Coseno Word2Vec
banco	institución financiera	asiento	0.5753	1.0
banco	institución financiera	cardumen	0.6603	1.0
banco	asiento	cardumen	0.6496	1.0
copa	trofeo	recipiente	0.5667	1.0
vela	de barco	cirio	0.7999	1.0

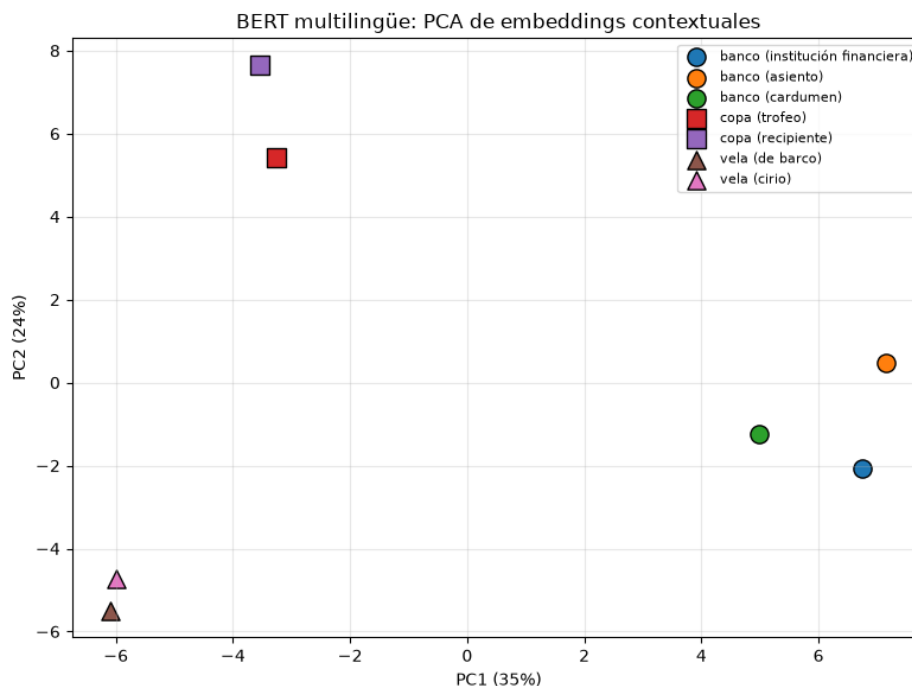
Con Word2Vec la similitud es 1.0 en todos los pares, esto es una consecuencia de cómo funciona el modelo. Word2Vec es estático y guarda un solo vector por palabra, así que “banco” tiene el mismo vector sin importar la oración en la que aparece. No puede distinguir entre diferentes sentidos semánticos.

Los vecinos más cercanos muestran qué sentido domina ese vector. Para “banco” son “hipotecario”, “bancos” o “banex”, todos del sentido financiero. Para “copa” son “supercopa”, “recopa” o “libertadores”, del sentido de trofeo. Para “vela” son términos de navegación como “envergá”, “envergada” o “gennaker”. Como el vector se aprende de todas las apariciones de la palabra en Wikipedia, queda dominado por el sentido más frecuente. Los sentidos menos comunes, como banco de asiento o vela de cirio, casi no se reflejan.

BERT, en cambio, sí separa los sentidos. Sus similitudes van de 0.57 a 0.80, claramente por debajo de 1, porque cada vector depende de la oración completa. El par con mayor similitud es el de “vela” con 0.80, lo que sugiere que BERT separó menos esos dos sentidos. Aun así, como hay una sola

oración por sentido las diferencias pequeñas entre pares pueden deberse a las oraciones específicas y no tanto al sentido. Se deberían de analizar más oraciones utilizando cada una de las palabras en sus diferentes sentidos.

(c) PCA de los embeddings de BERT



En la proyección PCA los puntos se agrupan por palabra y no por significado. Se forman tres grupos claros:

- Los tres “banco” a la derecha
- Los dos “copa” arriba a la izquierda
- Los dos “vela” abajo a la izquierda.

Dentro de cada grupo los distintos sentidos aparecen como puntos separados, así que BERT sí les asigna vectores diferentes. Sin embargo, la distancia entre sentidos de una misma palabra es mucho menor que la distancia entre palabras distintas. Es decir, la mayor fuente de variación en el espacio es qué palabra es, y el contexto solo la desplaza un poco dentro de su grupo.

Esto es consistente con las similitudes coseno de la tabla anterior. Los dos sentidos de “vela” quedan casi encima uno del otro, lo que coincide con su similitud de 0.80. En “banco”, el sentido de asiento queda algo apartado de los otros dos y es justo el par con menor similitud frente al sentido financiero. La visualización en dos dimensiones puede no ser la comparación más precisa, pero si podemos concluir que los contextos distintos sí se separan, pero dentro del espacio de la palabra y no como grupos independientes por significado.

Task 4.1

(a) Máscara causal y atención bidireccional

El objetivo de language modeling factoriza la probabilidad de una secuencia como

$$P(x_1, \dots, x_T) = \prod_t P(x_t | x_{<t})$$

Para que el modelo estime $P(x_t | x_{<t})$, la representación en la posición t solo puede depender de los tokens anteriores. La máscara causal logra esto sumando $-\infty$ a los scores de atención con $j > i$ antes del softmax, así que esos pesos quedan exactamente en 0. Esto es necesario porque en el entrenamiento todas las posiciones se calculan en paralelo con teacher forcing. Sin la máscara, la posición t podría ver el token x_{t+1} , que es su etiqueta.

Si GPT usara atención bidireccional en el preentrenamiento, predecir el siguiente token se volvería trivial. El modelo aprendería a copiar x_{t+1} desde la entrada y la pérdida caería casi a 0 sin que aprendiera mucho sobre el lenguaje. Además, el modelo ya no definiría una distribución $P(x_t | x_{<t})$ válida. El problema aparece en la generación, porque ahí el siguiente token todavía no existe. Lo que el modelo aprendió a hacer ya no sirve y hay una discrepancia entre cómo se entrenó y cómo se usa.

(b) Preentrenar y adaptar: GPT vs. BERT

En el Task 3, BERT dio vectores distintos para la misma palabra en contextos distintos con similitudes coseno entre 0.57 y 0.80, mientras que Word2Vec dio 1.0 en todos los casos. Esto pasa porque BERT calcula cada token atendiendo a la oración completa, tanto a la izquierda como a la derecha. Esa propiedad es útil para tareas de comprensión, como clasificación, desambiguación, similitud o extracción de entidades. GPT con atención causal, solo ha visto el contexto anterior cuando llega a “banco”. Su representación en ese punto no puede usar lo que viene después como “del parque” o “de peces”, que es justo lo que define el sentido. A cambio, GPT sí define $P(x_t | x_{<t})$ y puede generar texto, como hicimos en el Task 2.

El paradigma de preentrenar y adaptar resuelve esta tensión porque ambos modelos ya fueron preentrenados con grandes corpus. No hace falta entrenar ninguno desde cero, solo elegir el adecuado para la tarea. Si la tarea es entender o clasificar texto, se usa BERT y se le agrega una capa pequeña o se hace fine-tuning, partiendo de embeddings como los del Task 3. Si la tarea es generar o continuar texto, se usa GPT con prompting o fine-tuning. El costo grande de aprender el lenguaje se paga una sola vez en el preentrenamiento, y la adaptación a cada tarea es barata.

Nuestros resultados también muestran por qué la adaptación importa. La separación de significados en BERT fue parcial. Los dos sentidos de “vela” tuvieron una similitud de 0.80, y en el PCA los puntos se agruparon por palabra y no por significado. Por eso, para una desambiguación fina conviene hacer fine-tuning en lugar de usar los embeddings congelados tal cual.